LET'S LEARN MORE PYTHON

by Aditya Varma

Preface

Welcome to Let's Learn More Python

This book is written to help you learn Python as a skill, not as a subject.

Python is a language that shows how you can use simple English to program anything. It does not burden you with lots of syntax and meanings. Because of this, learning Python builds a great understanding to move on to further skills.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple programs that explain *why* something works, not just *how* to write it.

No prior programming knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Type the programs. Modify them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

-Aditya Varma

LICENSE & CREDITS

Credits:

Certain icons, illustrations, and portions of reference content are adapted from publicly available educational platforms, including the Coding X mobile application. Proper credit is given where applicable. All original rights remain with their respective creators, and this material is used solely for educational purposes.

Index

- Introduction
- Python Advance Functions
- Object Oriented Programming
- Extra OOPs
- Exception Handling
- File Handling
- Modules, Packages & Python Standard Library
- Extras

Introduction

Programming is not just about writing code, it is about learning how to think, solve problems, and build systems that work in the real world. In the earlier stages of learning Python, we focused on building a strong foundation. We explored the basics of the language, understood how variables store data, how operators perform calculations, and how control flow allows programs to make decisions. We practiced loops to repeat tasks efficiently and used data structures like lists, tuples, sets, and dictionaries to organize information.

We also learned how to write functions, which helped us break problems into smaller, manageable pieces. Concepts like arguments, return values, recursion, and lambda functions introduced us to more flexible and powerful ways of writing code. At this stage, we moved from simply writing instructions to designing logical solutions.

However, real-world programming goes far beyond these basics.

As programs grow larger and more complex, we need better ways to organize code, manage data, and build scalable systems. This is where advanced Python concepts come into play. In this book, we take the next step in your Python journey by exploring deeper and more powerful ideas.

We begin by strengthening our understanding of functions through topics like scope, closures, and function annotations. These concepts help us understand how Python manages memory and behaviour internally. From there, we move into Object-Oriented Programming (OOP), where we learn how to model real-world entities using classes and objects. Concepts like inheritance, polymorphism, encapsulation, and abstraction allow us to design structured and reusable systems.

As we progress, we explore advanced features such as decorators, iterators, and generators, which make Python more efficient and expressive. We also learn how Python handles errors and how to build programs that can deal with unexpected situations gracefully.

Another important part of this journey is working with files. We learn not only how to read and write data, but also how to handle large files, structured formats like CSV and JSON, and real-world data. This helps us understand how applications store and process information beyond memory.

Finally, we move into writing professional Python code by learning about modules, packages, and the Python Standard Library. These tools help us organize large projects, reuse existing code, and build applications in a clean and scalable way.

This book is designed to transform your understanding of Python, from writing simple scripts to building structured, efficient, and real-world applications. By the end of this journey, you will not only know *how* to write code, but also *how to design it thoughtfully*.

Let's move forward and explore the deeper side of Python.

Python Advance Functions

Modules, Packages & Standard Library

Till now, all programs were written in a single file. That works for small problems, but as programs grow, managing everything in one file becomes messy and difficult.

In real-world applications:

- Code is divided into multiple files
- Different parts handle different tasks
- Code is reused instead of rewritten

To achieve this, Python provides three important concepts:

- **Modules** → single Python files
- **Packages** → folders containing multiple modules
- **Standard Library** → built-in modules provided by Python

These concepts help in writing code that is organized, reusable, and easy to maintain.

What is a Module?

A module is simply a Python file (.py) that contains code such as functions, variables, or classes. Instead of writing everything in one file, we can split functionality into different modules. This makes:

- Code cleaner
- Easier to debug
- Reusable in multiple programs

What is a Package?

A package is a collection of modules organized inside a folder. In large projects, there can be many modules. Packages help group related modules together so the project structure remains clean and understandable. Think of it like:

- Module → a single file
- Package → a folder of related files

What is Standard Library?

Python comes with a huge collection of built-in modules known as the **Standard Library**. These modules provide ready-made solutions for common tasks like:

- Mathematical operations
- Random number generation
- File handling
- Working with dates and time

Instead of writing everything from scratch, we simply import and use them.

Real-World Thinking:

A real project looks like this:

```
project/
  |--- main.py
  |--- utils/
       |--- file_utils.py
       |--- math_utils.py
```

Instead of one giant file, everything is neatly divided. Modules, packages, and libraries help transform simple scripts into structured applications. They make code reusable, organized, and scalable, which is essential for professional development.

Lesson Program: Modules, Packages & Library in python.

Source Code:

IntroModulePackageLibrary.py

```python
# MODULES, PACKAGES & LIBRARY

print("\n===== USING MODULE =====")

import math

print("Square root:", math.sqrt(25))
print("Pi value:", math.pi)


print("\n===== ANOTHER MODULE =====")

import random

print("Random number:", random.randint( a: 1,  b: 10))

print("\n===== WHAT IS A PACKAGE =====")
print("A package = folder of modules (concept)")

print("\n===== STANDARD LIBRARY =====")
print("Examples: math, random, json, csv")

print("\n===== PRACTICAL IDEA =====")
print("""
Instead of writing logic again:
✓ Use modules
✓ Organize code in files
✓ Build clean structure
""")
```

Modules in Python

A module is simply a Python file (.py) that contains code such as functions, variables, or classes. Instead of writing all logic in one file, we split the program into multiple modules. This approach becomes necessary as programs grow larger. A single file quickly becomes difficult to read, debug, and maintain. By dividing code into modules, each file can focus on a specific task.

For example:

- A calculator module handles calculations
- A file module handles file operations
- A main file controls program flow

This separation makes code:

- Easier to understand
- Reusable across different programs
- Easier to test and debug

How Modules Work

Once a module is created, we can use it in another file using the import statement. Python gives multiple ways to import modules depending on how we want to use them. Each style has its own use case, helping balance readability and convenience.

Special Concept: `__name__`

Every Python file has a built-in variable called `__name__`.

- When the file is run directly → `__name__ = "__main__"`
- When the file is imported → `__name__ = module_name`

This helps control which part of the code should execute automatically and which part should only run when testing.

Lesson Program: Modules in python.

Source Code:

Modules.py

```python
# MODULES IN PYTHON

print("\n===== USING MODULE =====")

import math
print("Square root:", math.sqrt(16))

print("\n===== DIFFERENT IMPORT STYLES =====")

from math import sqrt
print("From import:", sqrt(25))

import math as m
print("Alias:", m.pi)

print("\n===== MULTIPLE IMPORT =====")

from math import sqrt, pi
print("Values:", sqrt(36), pi)

print("\n===== CUSTOM MODULE (CONCEPT) =====")
print("import calculator → calculator.add(2, 3)")

print("\n===== __name__ DEMO =====")

def test():  1 usage
    print("Test function executed")

if __name__ == "__main__":
    print("Running directly")
```

Packages in Python

As programs grow, even modules are not enough. You may end up with dozens of .py files, and managing them in a single folder becomes confusing.

This is where packages come in. A package is simply a folder that contains multiple modules, used to organize related functionality together. It helps structure large projects in a logical and readable way.

For example, instead of keeping everything in one place, we can group code like this:

- Math-related functions → math_utils.py
- File operations → file_utils.py
- Both inside a utils package

This makes the codebase:

- Easier to navigate
- Easier to maintain
- Easier to scale

How Packages Work

A package is a folder that usually contains:

- Python files (.py) → modules
- A special file __init__.py

The __init__.py file tells Python that this folder should be treated as a package. (In modern Python, it can be optional, but conceptually it's important for understanding.)

Importing from Packages

Once a package is created, we can import modules from it using:

- Direct import
- From-import style

This allows us to access specific functionality without cluttering the main file.

Lesson Program: Packages in python.

Source Code:

Packages.py

```python
# PACKAGES IN PYTHON

print("\n===== PACKAGE CONCEPT =====")
print("A package = folder of modules")


print("\n===== IMPORTING FROM PACKAGE =====")

print("from utils.math_utils import add")
print("OR")
print("import utils.math_utils")


print("\n===== USING IMPORTED FUNCTION =====")

# Simulating usage
def add(a, b):  1 usage
    return a + b

print("Addition:", add( a: 2,  b: 3))


print("\n===== WHY PACKAGES =====")
print("""
✓ Organize code
✓ Group related modules
✓ Avoid confusion
""")


print("\n===== PROJECT STRUCTURE =====")
print("""
project/
    main.py
    utils/
        math_utils.py
        file_utils.py
""")
```

Python Standard Library

Python comes with a large collection of built-in modules known as the **Standard Library**. These modules provide ready-made functionality for common tasks, so developers do not have to write everything from scratch.

Instead of manually coding solutions for things like file handling, random number generation, or date calculations, we can simply import and use these modules.This makes development:

- Faster
- More reliable
- Less error-prone

A key mindset in programming is:

👉 *Don't reinvent the wheel — use existing tools whenever possible.*

Standard Library vs External Libraries

- **Standard Library** → Comes with Python (no installation needed)
- **External Libraries** → Installed using pip

For example:

- math, random, os → already available
- requests, numpy → need installation

Commonly Used Modules

Some important standard library modules include:

- os → interacting with operating system
- sys → system-level operations
- math → mathematical functions
- random → random values
- datetime → date and time handling

Each module is designed for a specific purpose, making code modular and clean.

Basic modules like math and random solve common problems, but real-world programming often requires more powerful and efficient tools.

Python provides advanced modules in its standard library that help:

- Work with complex data structures
- Perform efficient looping and combinations
- Apply functional programming techniques

These modules reduce code complexity and improve performance significantly.

Key Modules Covered

1. collections

The collections module provides advanced data structures beyond basic lists, dictionaries, and tuples.

It is especially useful when:

- Counting elements
- Grouping data
- Providing default values

For example:

- Counter → counts occurrences
- defaultdict → avoids key errors with default values

2. itertools

The itertools module provides tools for efficient iteration.

It is useful when:

- Generating combinations or permutations
- Working with sequences
- Creating infinite iterators

These tools are memory-efficient and commonly used in algorithms and data processing.

3. functools

The functools module supports functional programming.

It is used for:

- Aggregating data (reduce)
- Optimizing functions
- Writing cleaner logic

It helps reduce repetitive code and express logic more compactly.

Lesson Program: Library in python.

Source Code:

StandardLibrary.py

```python
# PYTHON STANDARD LIBRARY

print("\n===== USING STANDARD LIBRARY =====")

import math
print("Square root:", math.sqrt(36))

print("\n===== RANDOM MODULE =====")

import random
print("Random number:", random.randint( a: 1,  b: 10))

print("\n===== DATETIME MODULE =====")

import datetime
print("Current time:", datetime.datetime.now())

print("\n===== OS MODULE =====")

import os
print("Current directory:", os.getcwd())

print("\n===== SYS MODULE =====")

import sys
print("Python version:", sys.version)

# ADVANCED STANDARD LIBRARY

print("\n===== COLLECTIONS =====")

from collections import Counter, defaultdict

data = ["a", "b", "a", "c"]
print("Count:", Counter(data))

d = defaultdict(int)
d["x"] += 1
print("DefaultDict:", d)

print("\n===== ITERTOOLS =====")

import itertools

print("Combinations:", list(itertools.combinations( iterable: [1, 2, 3],  r: 2)))
print("Permutations:", list(itertools.permutations( iterable: [1, 2],  r: 2)))

print("\n===== FUNCTOOLS =====")

from functools import reduce

nums = [1, 2, 3, 4]
print("Sum:", reduce(lambda x, y: x + y, nums))

print("\n===== PRACTICAL =====")

text = "python is powerful python"
print("Word Count:", Counter(text.split()))
```

External Libraries in Python

So far, we have used Python's built-in features and the standard library. However, in real-world development, many complex problems have already been solved by other developers. Instead of writing everything from scratch, Python allows us to use these ready-made solutions in the form of external libraries.

An external library is a collection of pre-written code that is not included with Python by default but can be added to a project when needed. These libraries are created and maintained by developers across the world and cover a wide range of domains such as web development, data analysis, machine learning, and automation.

Using external libraries significantly speeds up development. Instead of spending time building complex functionality, we can focus on solving the actual problem. This not only saves effort but also reduces errors, since these libraries are usually well-tested and widely used.

Role of pip:

To use external libraries, Python provides a tool called pip, which acts as a package manager. It allows us to install, update, and remove libraries easily. You can think of pip as an app store for Python, where you can download and manage packages required for your project.

The general workflow is simple:

1. Install the library using pip
2. Import it into your program
3. Use its features

When to Use External Libraries

External libraries should be used when:

- The problem is complex and already solved
- Performance and optimization are required
- Industry-standard tools are needed

However, for simple tasks or while learning fundamentals, it is better to write logic manually to build understanding.

Lesson Program: External Library in python.

Source Code:

ExternalLibrary.py

```python
# EXTERNAL LIBRARIES IN PYTHON

print("\n===== USING EXTERNAL LIBRARIES =====")

print("""
Step 1: Install (in terminal)
pip install requests
""")


print("\n===== IMPORTING LIBRARY =====")

# Example usage (simulation)
print("import requests → used for API calls")


print("\n===== ANOTHER EXAMPLE =====")

print("""
pip install numpy
import numpy as np
""")


print("\n===== WHY USE LIBRARIES =====")
print("""
✓ Save time
✓ Use ready-made solutions
✓ Build real-world applications faster
""")


print("\n===== REAL USE CASE =====")
print("""
requests → web/API data
numpy → numerical computing
pandas → data analysis
""")
```

Modules, Packages & Libraries Summary

Modules are individual Python files (.py) that contain functions, variables, and classes; they help organize and reuse code.

- Modules are used using the import statement:
  - import module
  - from module import function
  - import module as alias
- __name__ == "__main__" is used to control whether code runs when the file is executed directly or imported.

Packages are folders that contain multiple modules; they help organize large codebases into logical groups.

- A package typically contains:
  - Multiple .py files (modules)
  - __init__.py (marks directory as a package)
- Modules inside packages are accessed using:
  - from package.module import function
  - import package.module
- Packages improve:
  - Code structure
  - Readability
  - Maintainability
  - Avoid naming conflicts

Python Standard Library is a collection of built-in modules that come with Python and solve common problems.

- Common standard library modules:
  - math → calculations
  - random → random values
  - datetime → date & time
  - os → system operations
  - sys → system interaction
- Standard library helps:
  - Avoid rewriting code
  - Save time
  - Use reliable, tested solutions

Advanced Standard Library modules provide powerful tools:

- collections → advanced data structures (Counter, defaultdict)
- itertools → efficient looping (combinations, permutations)
- functools → functional tools (reduce)

External Libraries are not built-in and must be installed using pip.

- Installation command:
  - pip install library_name
- Examples of external libraries:
  - requests → API calls
  - numpy → numerical computing
  - pandas → data analysis
  - matplotlib → visualization
- Workflow for external libraries:
  - Install → Import → Use
- External libraries:
  - Speed up development
  - Provide advanced features
  - Are widely used in real-world applications